

实验报告

学生姓名：	汤承洲	学 号：	20233902058
专 业：	电子信息工程	年级、班级：	23 级 2 班
课程名称：	控制工程	实验项目：	控制系统现代分析
实验类型：	☒ 验证 ☐ 设计 ☐ 综合	实验时间：	2026 年 6 月 2 日
实验指导老师：	熊爱民	实验评分：	

1. 实验目的

1. 掌握如何使用 MATLAB 进行系统的稳定性分析，包括代数法（Routh 判据、求特征根）和频域法（Bode 图）两种方法。
2. 掌握如何使用 MATLAB 进行系统的能控性、能观性分析，包括能控性矩阵、能观性矩阵以及 Gramian 矩阵方法。
3. 掌握如何使用 MATLAB 进行离散系统分析，研究不同采样率对离散化精度的影响。

2. 实验原理

2.1. 系统稳定性分析原理

控制系统稳定性的判定是控制理论的核心问题之一。本实验采用代数法和频域法两种方法进行稳定性分析。

代数法稳定性判据：通过求解闭环特征方程的根来判断系统稳定性。若所有特征根均具有负实部（位于左半 s 平面），则系统稳定。Routh-Hurwitz 判据通过构造 Routh 表，检查第一列系数的符号变化次数，判断特征根中位于右半 s 平面的个数，无需直接求解特征方程。

Bode 图法稳定性判据：基于 Nyquist 稳定判据，通过开环系统的频率响应特性判断闭环稳定性。幅值裕度（Gain Margin）和相角裕度（Phase Margin）是衡

量系统相对稳定性的指标。对于最小相位系统，当幅值裕度 > 0 dB 且相角裕度 $> 0^\circ$ 时，闭环系统稳定。

2.2. 系统能控性与能观性分析原理

能控性 (**Controllability**): 若存在一个无约束的控制向量 $\mathbf{u}(t)$ ，能在有限时间 $[t_0, t_f]$ 内将系统从任意初始状态 $\mathbf{x}(t_0)$ 转移到任意终态 $\mathbf{x}(t_f)$ ，则称系统状态完全能控。能控性判据为能控性矩阵 $\mathbf{C}_o = [\mathbf{B}, \mathbf{AB}, \mathbf{A}^2\mathbf{B}, \dots, \mathbf{A}^{n-1}\mathbf{B}]$ 满秩 ($\text{rank}(\mathbf{C}_o) = n$)。

能观性 (**Observability**): 若在有限时间 $[t_0, t_f]$ 内，通过测量输出 $\mathbf{y}(t)$ 能唯一确定系统的初始状态 $\mathbf{x}(t_0)$ ，则称系统状态完全能观。能观性判据为能观性矩阵 $\mathbf{O}_b = [\mathbf{C}, \mathbf{CA}, \mathbf{CA}^2, \dots, \mathbf{CA}^{n-1}]^T$ 满秩 ($\text{rank}(\mathbf{O}_b) = n$)。

对于稳定系统，还可以通过求解 Lyapunov 方程得到能控性 Gramian 矩阵 $\mathbf{W}_c$ 和能观性 Gramian 矩阵 $\mathbf{W}_o$ ，其秩同样可用于判断能控性和能观性。

2.3. 采样率影响分析原理

数字控制系统需要对连续信号进行采样和离散化。根据香农采样定理，采样频率 f_s 必须大于信号最高频率的两倍 (即 $f_s > 2f_{\max}$)，才能从采样信号中无失真地恢复原始信号。本实验通过零阶保持器 (ZOH) 方法将连续系统离散化，在不同采样率下比较离散系统与连续系统的频率响应差异，分析采样率对离散化精度的影响。

3. 实验内容

3.1. Task 01: 系统稳定性分析

3.1.1 代数法稳定性判据

已知系统的开环传递函数为：

$$G(s) = 100 \frac{s+3}{s(s+1)(s+10)}$$

对系统闭环判别其稳定性。闭环特征多项式为：

$$1 + G(s) = 0 \implies s(s+1)(s+10) + 100(s+3) = 0$$

展开得：

$$s^3 + 11s^2 + 110s + 300 = 0$$

方法一：求特征根。求解闭环特征多项式，得到特征根：

- $-3.7007 + 8.3469i$
- $-3.7007 - 8.3469i$
- -3.5986

三个根均具有负实部，全部位于左半 s 平面，故闭环系统稳定。

方法二：**Routh** 判据。构造 **Routh** 表如下：

s^3	1	110
s^2	11	300
s^1	82.7273	0
s^0	300	0

Routh 表第一列系数为：1、11、82.7273、300，全部为正数，无符号变化，故闭环系统稳定。两种方法结论一致。

3.1.2 Bode 图法判断系统稳定性

已知两个单位负反馈系统的开环传递函数分别为：

$$G_1(s) = \frac{3}{s^3 + 5s^2 + 3s}, \quad G_2(s) = \frac{3}{s^3 + 5s^2 - 3s}$$

用 **Bode** 图法判断系统闭环的稳定性。

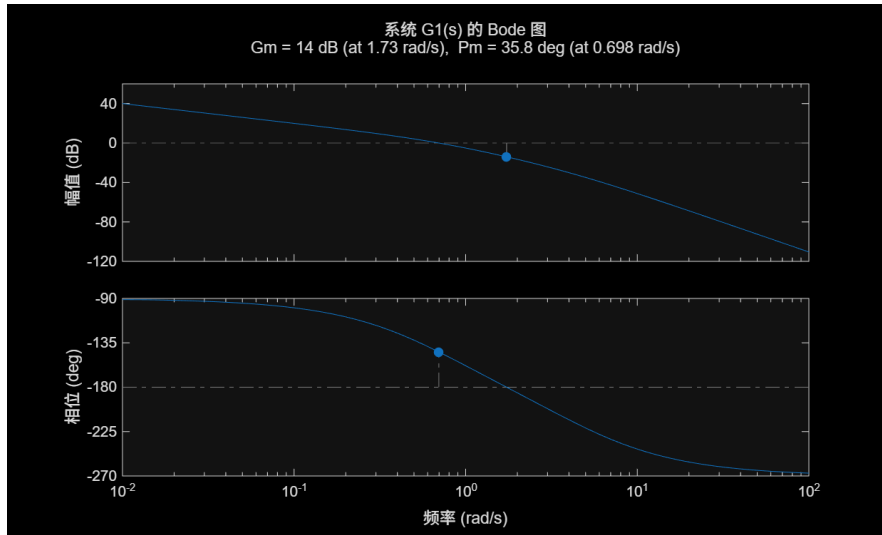


图 1: 系统 G1(s) 的 Bode 图

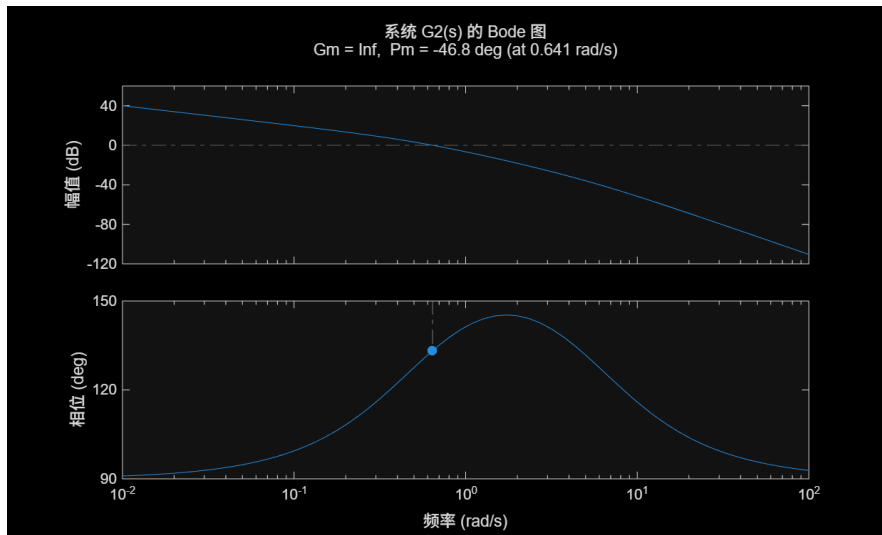


图 2: 系统 G2(s) 的 Bode 图

Bode 图分析结果如表 1 所示。

表 1: Bode 图法稳定性分析结果			
系统	幅值裕度 (dB)	相角裕度 (deg)	稳定性结论
G1(s)	13.98	35.77	闭环稳定
G2(s)	Inf	-46.78	闭环不稳定

G1(s) 的幅值裕度为 13.98 dB，相角裕度为 35.77°，均为正值，故闭环系统稳定。G2(s) 的相角裕度为 -46.78°（负值），故闭环系统不稳定。该结果与通过

开环传递函数中一次项系数符号的直接判断一致：G1(s) 分母中 s 项系数 +3 表示系统稳定，G2(s) 分母中 s 项系数 -3（存在右半平面极点）表示系统不稳定。

3.2. Task 02: 系统能控性与能观性分析

已知连续系统的传递函数模型：

$$G(s) = \frac{s + a}{s^3 + 10s^2 + 27s + 8}$$

当 $a = -2, 0, +2$ 时，判别系统的能控性与能观性。

系统状态空间实现采用控制器标准型（companion form）：

$$\mathbf{A} = \begin{bmatrix} -10 & -27 & -8 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 0 & 1 & a \end{bmatrix}, \quad D = 0$$

系统极点为 -4.8315、-4.8315、-0.3369，全部在左半平面，系统稳定。

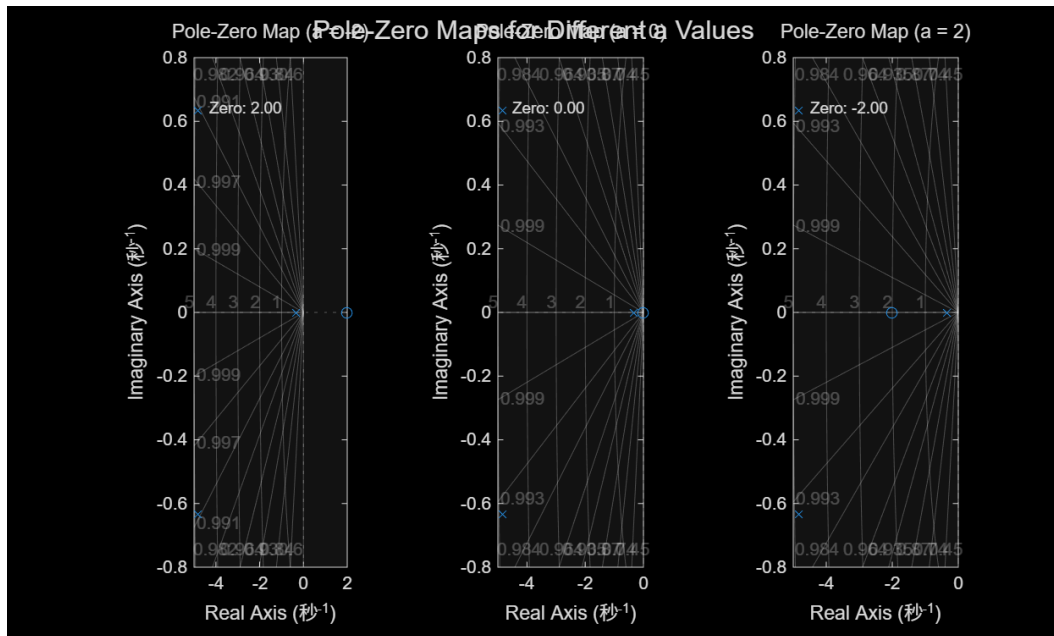


图 3: 不同 a 值下的零极点图

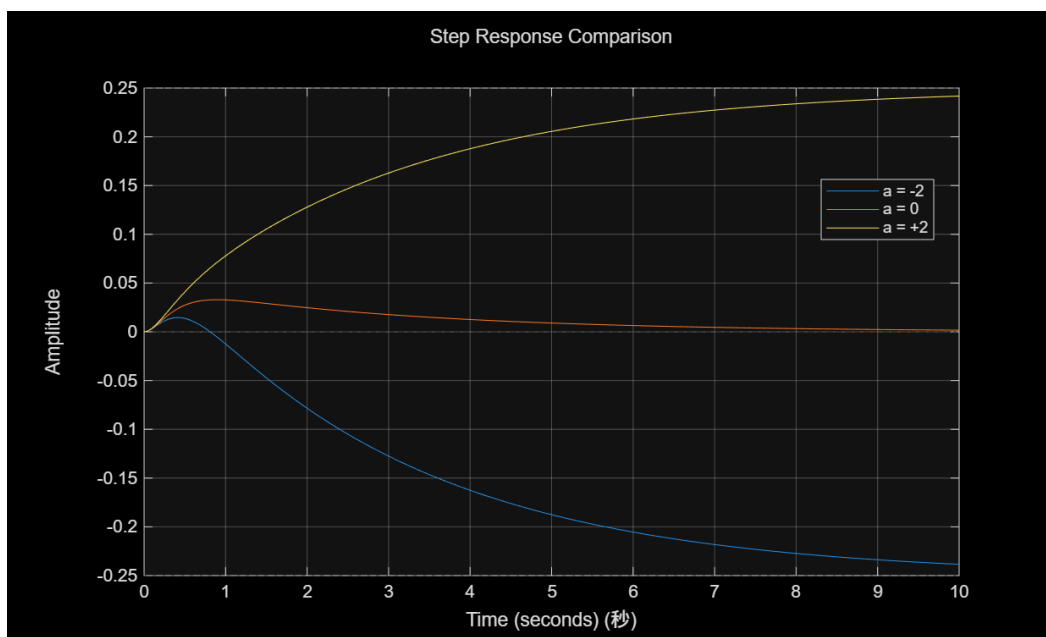


图 4: 不同 a 值下的阶跃响应对比

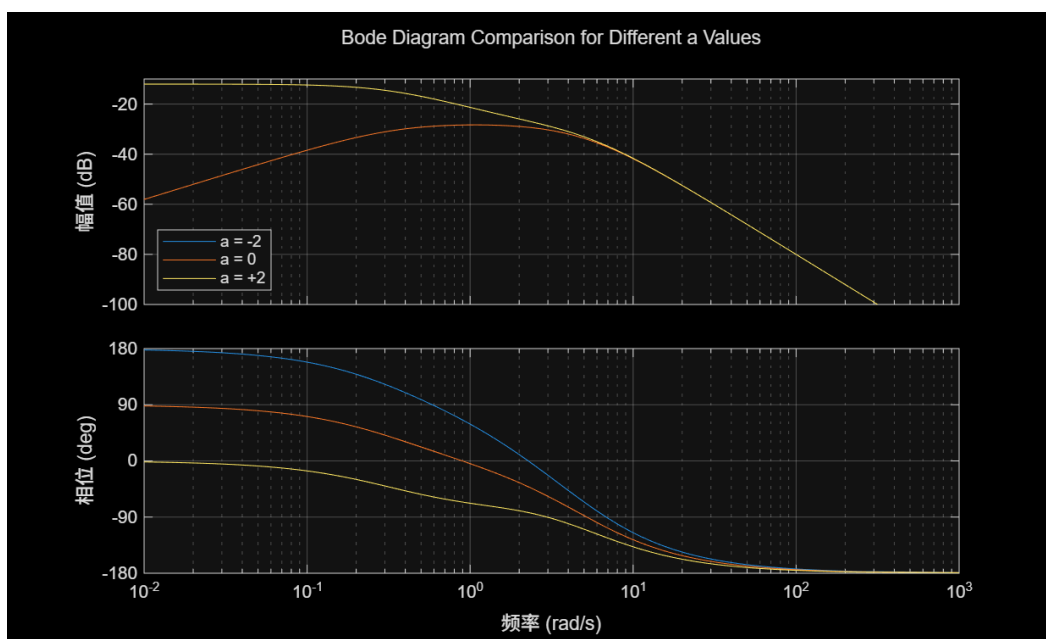


图 5: 不同 a 值下的 Bode 图对比

表 2: 能控性与能观性分析结果汇总

a	$\text{rank}(\text{Co})$	$\text{rank}(\text{Ob})$	能控性 (矩阵)	能观性 (矩阵)	Gramian 验证
-2	3	3	完全能控	完全能观	一致
0	3	3	完全能控	完全能观	一致
+2	3	3	完全能控	完全能观	一致

能控性矩阵为:

$$\mathbf{C}_o = [\mathbf{B}, \mathbf{AB}, \mathbf{A}^2\mathbf{B}] = \begin{bmatrix} 1 & -10 & 73 \\ 0 & 1 & -10 \\ 0 & 0 & 1 \end{bmatrix}$$

$\text{rank}(\mathbf{C}_o) = 3 = n$, 三种参数下系统均为完全能控。

能观性矩阵随 a 变化:

$$\mathbf{O}_b(a = -2) = \begin{bmatrix} 0 & 1 & -2 \\ 1 & -2 & 0 \\ -12 & -27 & -8 \end{bmatrix}, \quad \mathbf{O}_b(a = 0) = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ -10 & -27 & -8 \end{bmatrix}, \quad \mathbf{O}_b(a = +2) = \begin{bmatrix} 0 & 1 \\ 1 & 2 \\ -8 & -27 \end{bmatrix}$$

均满秩 ($\text{rank}(\mathbf{O}_b) = 3$), 三种参数下系统均为完全能观。

Gramian 矩阵验证结果与能控性/能观性矩阵判据一致, 均满足 $\text{rank}(\mathbf{W}_c) = \text{rank}(\mathbf{W}_o) = 3$ 。

实验表明, 参数 a 的变化影响零点的位置, 从而影响系统的动态响应特性 (如阶跃响应的初始值和稳态值), 但不改变系统的能控性和能观性结构。

3.3. Task 03: 采样率对离散化的影响

对 Task 01 中 $G_2(s)$ 的穿越频率进行分析, 并用不同采样率将其离散化。

$$G_2(s) = \frac{3}{s^3 + 5s^2 - 3s}$$

增益穿越频率 (Gain Crossover Frequency):

- $w_c = 0.641 \text{ rad/s}$
- $f_1 = w_c/(2\pi) = 0.102 \text{ Hz}$

采用零阶保持器 (ZOH) 方法, 以 $f = f_1, 3f_1, 10f_1$ 三种采样率将连续系统离散化, 绘制 Bode 图和 Nyquist 图, 并分析离散化精度。

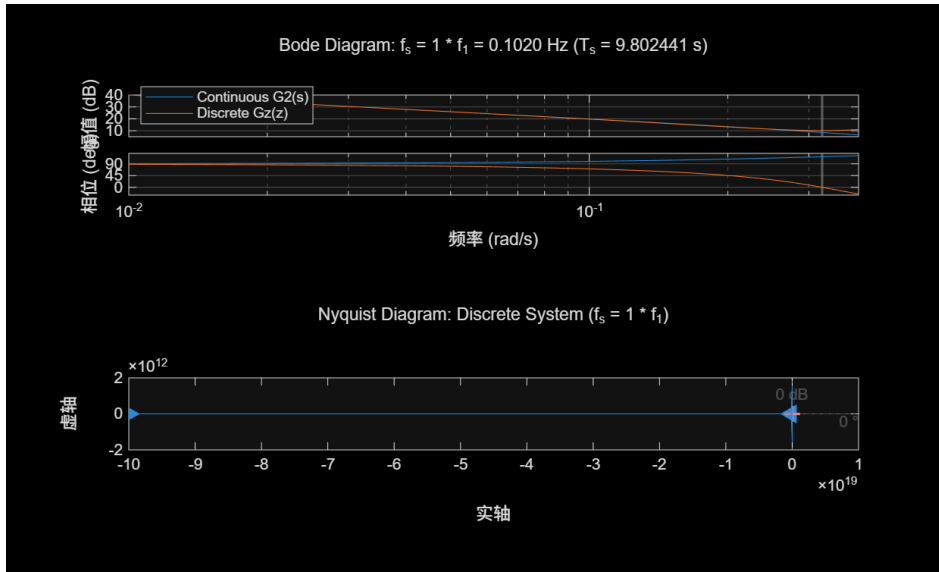


图 6: $f = f_1$ ($T_s = 9.802 \text{ s}$) 时的 Bode 图和 Nyquist 图

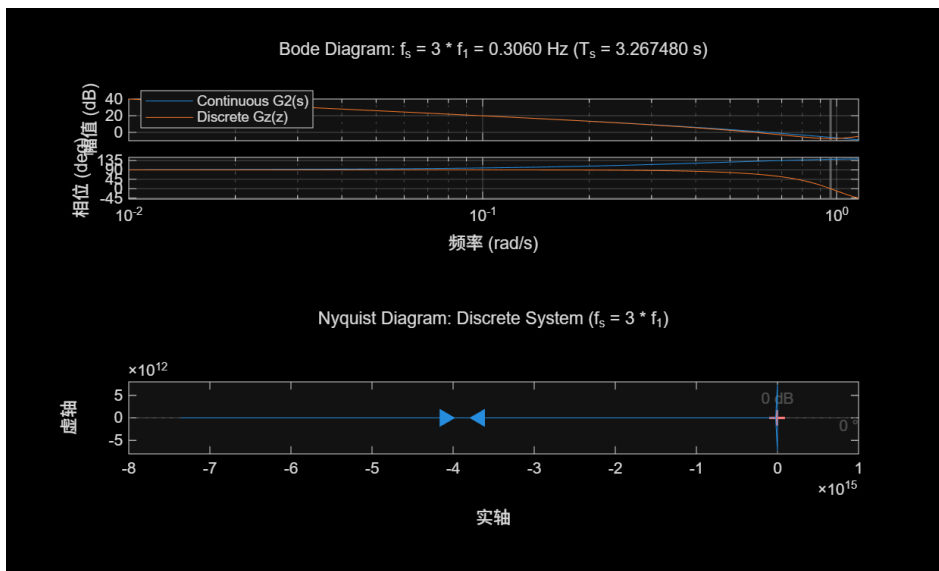


图 7: $f = 3f_1$ ($T_s = 3.267 \text{ s}$) 时的 Bode 图和 Nyquist 图

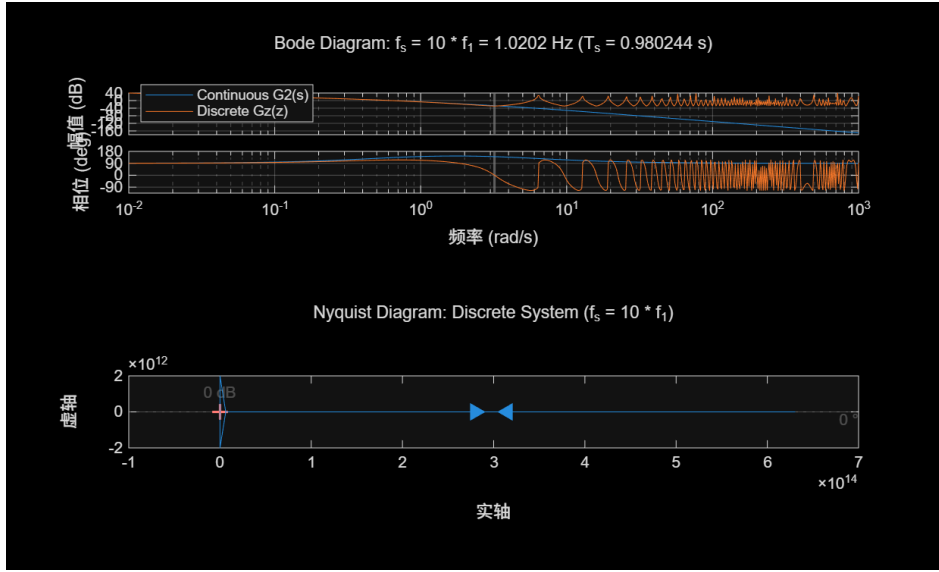


图 8: $f = 10f_1$ ($T_s = 0.980$ s) 时的 Bode 图和 Nyquist 图

表 3: 不同采样率下离散系统与连续系统的对比

采样率	T_s (s)	幅值裕度 (dB)	相角裕度 (deg)	幅值误差 (dB)	相角误差 (deg)
连续	N/A	Inf	-46.78	—	—
$f = f_1$	9.802	Inf	Inf	332.05	313.22
$f = 3f_1$	3.267	Inf	-109.75	1.51	68.82
$f = 10f_1$	0.980	Inf	-64.90	0.14	18.08

结果分析:

1. $f = f_1$ (最低采样率): 采样频率等于穿越频率, Nyquist 频率 (0.05 Hz) 低于信号频率成分, 严重违反香农采样定理。离散系统完全失真, 幅值误差高达 332.05 dB, 频率响应无法反映连续系统的特性。
2. $f = 3f_1$: Nyquist 频率 (0.15 Hz) 超过穿越频率, 满足基本 Nyquist 准则。但离散化精度仍有限, 幅值误差 1.51 dB, 相角误差 68.82°, 低频段匹配尚可。
3. $f = 10f_1$: 高采样率下离散系统与连续系统的频率响应高度吻合, 幅值误差仅 0.14 dB, 相角误差 18.08°。零阶保持器引入的相位滞后在感兴趣的频率范围内影响较小。

结论: 采样率越高, 离散系统越精确地逼近连续系统的频率响应。建议在实

际数字控制系统实现中，采样率至少取系统穿越频率的 10 倍，以保证足够的离散化精度和控制性能。

4. 实验总结

本次实验围绕控制系统现代分析方法，系统地进行了稳定性分析、能控性能观性分析以及采样率影响分析三个方面的实践。

稳定性分析方面：

1. 通过代数法（Routh 判据和求特征根）和频域法（Bode 图）两种方法对系统进行稳定性判别，两种方法结论一致，验证了代数判据与频域判据的等价性。
2. 对于 $G1(s) = 3/(s^3 + 5s^2 + 3s)$ ，幅值裕度 13.98 dB、相角裕度 35.77° ，系统稳定；对于 $G2(s) = 3/(s^3 + 5s^2 - 3s)$ ，相角裕度 -46.78° ，系统不稳定。 $G2(s)$ 的不稳定性源于其开环传递函数包含一个右半平面极点（ s 项系数为 -3 ）。

能控性与能观性分析方面：

1. 对传递函数 $G(s) = (s + a)/(s^3 + 10s^2 + 27s + 8)$ 在 $a = -2, 0, +2$ 三种参数下进行了分析，能控性矩阵和能观性矩阵的秩均为 3（满秩），Gramian 矩阵验证一致。
2. 三种参数下系统均为完全能控且完全能观。参数 a 的变化影响零点的位置和系统动态响应特性（阶跃响应、Bode 图），但不改变能控性和能观性结构。
3. 能控性矩阵与系统矩阵 A 和输入矩阵 B 有关，与输出矩阵 C （参数 a 所在位置）无关，因此参数 a 不影响能控性。而能观性矩阵虽与 C 有关，但三种参数下均满秩，说明该系统的结构固有能观。

采样率影响分析方面：

1. 对不稳定系统 $G_2(s)$ 进行了从连续到离散的转换，研究了 $f = f_1$ 、 $3f_1$ 、 $10f_1$ 三种采样率的影响。
2. $f = f_1$ 时离散模型严重失真； $f = 3f_1$ 时精度有所改善但仍有较大相位误差； $f = 10f_1$ 时离散模型与连续模型高度吻合。
3. 推荐实际应用中采样率至少取系统穿越频率的 10 倍，以保证数字控制系统的性能。

本次实验加深了对控制系统稳定性、能控能观性以及数字离散化理论的理解，掌握了 MATLAB 控制系统工具箱在系统分析中的综合运用方法。

5. 实验源码

5.1. Task 01: 系统稳定性分析

```

1 % 系统稳定性分析
2 clc;
3 clear;
4 close all;
5 diary('results/logs/Task01_output.log');
6
7 %% 1.1 代数法稳定性判据
8 % 已知开环传递函数  $G(s) = 100*(s+3) / [s(s+1)(s+10)]$ 
9 % 闭环特征多项式  $= 1 + G(s) = 0$ 
10
11 % 开环分母展开
12 den_open = conv([1 0], conv([1 1], [1 10]));
13 num_open = 100 * [1 3];
14
15 % 将分子补零到与分母相同长度
16 n_den = length(den_open);
17 n_num = length(num_open);
18 num_open = [zeros(1, n_den - n_num), num_open];
19
20 % 闭环特征多项式 = 分母 + 分子
21 den_closed = den_open + num_open;
22
23 % 方法一：求分母多项式的根
24 roots_closed = roots(den_closed);
25 disp('=== 代数法稳定性判据 ===');
26 disp('方法一：求分母多项式的根');

```

```

27 disp(roots_closed);
28
29 % 判断稳定性
30 if all(real(roots_closed) < 0)
31     disp('结论：闭环系统稳定');
32 else
33     disp('结论：闭环系统不稳定');
34 end
35
36 % 方法二：Routh 判据
37 r = routh(den_closed, 1e-6);
38 disp('方法二：Routh 函数');
39 disp('Routh 表：');
40 disp(r);
41 disp('Routh 表第一列：');
42 disp(r(:, 1));
43
44 % 检查第一列是否有符号变化
45 first_col = r(:, 1);
46 sign_changes = 0;
47 for i = 1:length(first_col) - 1
48     if first_col(i) * first_col(i + 1) < 0
49         sign_changes = sign_changes + 1;
50     end
51 end
52 fprintf('Routh 表第一列符号变化次数：%d\n', sign_changes);
53 if sign_changes == 0 && all(first_col > 0)
54     fprintf('结论：闭环系统稳定\n');
55 else
56     fprintf('结论：闭环系统不稳定，有 %d 个右半平面极点\n',
57         sign_changes);
58 end
59
60 %% 1.2 Bode图法判断系统稳定性
61 % 两个单位负反馈系统的开环传递函数
62
63 %  $G1(s) = 3 / (s^3 + 5s^2 + 3s)$ 
64 den_G1 = [1 5 3 0];
65 num_G1 = [3];
66
67 %  $G2(s) = 3 / (s^3 + 5s^2 - 3s)$ 
68 den_G2 = [1 5 -3 0];
69 num_G2 = [3];
70
71 disp('=====');
72 disp('=== Bode图法稳定性判据 ===');
73
74 % 绘制 G1 的 Bode 图
75 figure(1);

```

```

75 margin(tf(num_G1, den_G1));
76 title('系统 G1(s) 的 Bode 图');
77 saveas(gcf, 'results/figures/Task01_Figure_01.png');
78
79 [Gm1, Pm1, Wcg1, Wcp1] = margin(tf(num_G1, den_G1));
80 fprintf('\nG1(s): 幅值裕度 = %.2f dB, 相角裕度 = %.2f deg\n', ...
81         20*log10(Gm1), Pm1);
82
83 if Pm1 > 0 && Gm1 > 1
84     fprintf('结论: G1(s) 闭环系统稳定.\n');
85 else
86     fprintf('结论: G1(s) 闭环系统不稳定.\n');
87 end
88
89 % 绘制 G2 的 Bode 图
90 figure(2);
91 margin(tf(num_G2, den_G2));
92 title('系统 G2(s) 的 Bode 图');
93 saveas(gcf, 'results/figures/Task01_Figure_02.png');
94
95 [Gm2, Pm2, Wcg2, Wcp2] = margin(tf(num_G2, den_G2));
96 fprintf('\nG2(s): 幅值裕度 = %.2f dB, 相角裕度 = %.2f deg\n', ...
97         20*log10(Gm2), Pm2);
98
99 if Pm2 > 0 && Gm2 > 1
100     fprintf('结论: G2(s) 闭环系统稳定.\n');
101 else
102     fprintf('结论: G2(s) 闭环系统不稳定.\n');
103 end
104
105 diary off;
106 disp('Task 1 完成。');

```

Listing 1: Routh 判据辅助函数

```

1 function ra = routh(poli, epsilon)
2 % Routh-Hurwitz 稳定性判据数组计算
3 % poli: 多项式系数向量 (降幂排列)
4 % epsilon: 行首零元素的替代值
5
6 dim = size(poli);
7 coeff = dim(2);
8 ra = zeros(coeff, ceil(coeff / 2));
9
10 % 组装第 1 行和第 2 行
11 for i = 1 : coeff
12     ra(2 - rem(i, 2), ceil(i / 2)) = poli(i);
13 end
14

```

```

15 rows = coeff - 2;
16 index = zeros(rows, 1);
17
18 % 从底部到顶部形成索引向量
19 for i = 1 : rows
20     index(rows - i + 1) = ceil(i / 2);
21 end
22
23 % 从第 3 行到最后一行
24 for i = 3 : coeff
25     if all(ra(i - 1, :) == 0) % 整行全为零
26         a = coeff - i + 2;
27         b = ceil(a / 2) - rem(a, 2) + 1;
28         temp1 = ra(i - 2, 1 : b);
29         temp2 = a : -2 : 0;
30         ra(i - 1, 1 : b) = temp1 .* temp2;
31     elseif ra(i - 1, 1) == 0 % 行首元素为零
32         ra(i - 1, 1) = epsilon;
33     end
34
35     for j = 1 : index(i - 2)
36         ra(i, j) = -det([ra(i - 2, 1), ra(i - 2, j + 1); ...
37             ra(i - 1, 1), ra(i - 1, j + 1)]) / ra(i - 1, 1);
38     end
39 end
40 end

```

5.2. Task 02: 系统能控性与能观性分析

```

1 % System Controllability and Observability Analysis
2 clc;
3 clear;
4 close all;
5 diary('results/logs/Task02_output.log');
6
7 fprintf('
8     =====\n');
9 fprintf('Task 2: System Controllability and Observability Analysis\n');
10 ;
11 fprintf('Transfer function: G(s) = (s + a) / (s^3 + 10s^2 + 27s + 8)\n');
12 ;
13 fprintf('a = -2, 0, +2\n');
14 fprintf('
15     =====\n');
16 ;
17 fprintf('a\n');
18 ;
19 a_values = [-2, 0, 2];

```

```

14 n = 3; % system order
15
16 % Store results for summary
17 results_data = [];
18
19 for idx = 1:length(a_values)
20     a = a_values(idx);
21     fprintf('===== a = %d =====\n', a);
22
23     % Numerator and denominator
24     num = [1, a];
25     den = [1, 10, 27, 8];
26
27     % Convert to state-space using tf2ss (controller canonical form)
28     [A, B, C, D] = tf2ss(num, den);
29
30     fprintf('State-space representation (controller canonical form):\n
31             ');
32     fprintf('A =\n');
33     disp(A);
34     fprintf('B =\n');
35     disp(B);
36     fprintf('C =\n');
37     disp(C);
38     fprintf('D = %d\n', D);
39
40     % --- Controllability Analysis ---
41     Co = ctrb(A, B);
42     rank_Co = rank(Co);
43     fprintf('Controllability matrix Co =\n');
44     disp(Co);
45     fprintf('Rank(Co) = %d (system order n = %d)\n', rank_Co, n);
46
47     if rank_Co == n
48         fprintf('>> System is CONTROLLABLE.\n');
49         controllable = true;
50     else
51         fprintf('>> System is NOT CONTROLLABLE.\n');
52         controllable = false;
53     end
54
55     % --- Observability Analysis ---
56     Ob = obsv(A, C);
57     rank_Ob = rank(Ob);
58     fprintf('\nObservability matrix Ob =\n');
59     disp(Ob);
60     fprintf('Rank(Ob) = %d (system order n = %d)\n', rank_Ob, n);
61
62     if rank_Ob == n

```

```

62         fprintf('>> System is OBSERVABLE.\n');
63         observable = true;
64     else
65         fprintf('>> System is NOT OBSERVABLE.\n');
66         observable = false;
67     end
68
69     % --- Verification using Gramians ---
70     fprintf('\n--- Gramian Verification ---\n');
71     sys = ss(A, B, C, D);
72     poles = eig(A);
73     fprintf('System poles: ');
74     fprintf('%0.4f ', real(poles));
75     fprintf('\n');
76
77     if all(real(poles) < -1e-10)
78         try
79             Wc = gram(sys, 'c');
80             Wo = gram(sys, 'o');
81             fprintf('Controllability Gramian Wc =\n');
82             disp(Wc);
83             fprintf('Observability Gramian Wo =\n');
84             disp(Wo);
85
86             sv_Wc = svd(Wc);
87             sv_Wo = svd(Wo);
88             fprintf('Singular values of Wc: ');
89             fprintf('%0.6e ', sv_Wc);
90             fprintf('\n');
91             fprintf('Singular values of Wo: ');
92             fprintf('%0.6e ', sv_Wo);
93             fprintf('\n');
94
95             rank_Wc = rank(Wc, 1e-6);
96             rank_Wo = rank(Wo, 1e-6);
97             fprintf('Rank(Wc) = %d, Rank(Wo) = %d\n', rank_Wc, rank_Wo
98                 );
99
100             if rank_Wc == n
101                 fprintf('Gramian verification: CONTROLLABLE.\n');
102             else
103                 fprintf('Gramian verification: NOT CONTROLLABLE.\n');
104             end
105             if rank_Wo == n
106                 fprintf('Gramian verification: OBSERVABLE.\n');
107             else
108                 fprintf('Gramian verification: NOT OBSERVABLE.\n');
109             end
110         catch ME

```



```

110         fprintf('Gramian computation failed: %s\n', ME.message);
111     end
112     else
113         fprintf('System has non-negative real poles; gramian skipped.\n');
114     end
115
116     results_data = [results_data; a, rank_Co, rank_Ob, ...
117         controllable, observable];
118     fprintf('\n');
119 end
120
121 % --- Summary Table ---
122 fprintf('
123     =====\n');
124 fprintf('SUMMARY OF RESULTS\n');
125 fprintf('
126     =====\n');
127 fprintf('  a    | rank(Co) | rank(Ob) | Controllable | Observable\n');
128 fprintf('-----+-----+-----+-----+\n');
129 for i = 1:size(results_data, 1)
130     c_str = 'Yes';
131     o_str = 'Yes';
132     if results_data(i, 4) == 0
133         c_str = 'No';
134     end
135     if results_data(i, 5) == 0
136         o_str = 'No';
137     end
138     fprintf('  %2d    |    %d    |    %d    |    %s    |    %s\n',
139         ...
140         results_data(i, 1), results_data(i, 2), ...
141         results_data(i, 3), c_str, o_str);
142 end
143 fprintf('
144     =====\n');
145 fprintf('Note: System order n = %d. Full rank (n) means controllable/
146     observable.\n', n);
147 fprintf('
148     =====\n');
149
150 % --- Pole-Zero Maps ---
151 figure(1);
152 for idx = 1:length(a_values)
153     a = a_values(idx);

```

```

148     subplot(1, 3, idx);
149     sys_tf = tf([1, a], [1, 10, 27, 8]);
150     pzmap(sys_tf);
151     title(sprintf('Pole-Zero Map (a = %d)', a));
152     xlabel('Real Axis');
153     ylabel('Imaginary Axis');
154     grid on;
155     [p, z] = pzmap(sys_tf);
156     text_x = xlim;
157     text_y = ylim;
158     if ~isempty(z)
159         text(text_x(1)+0.1*(text_x(2)-text_x(1)), ...
160             text_y(2)-0.1*(text_y(2)-text_y(1)), ...
161             sprintf('Zero: %.2f', real(z(1))), 'FontSize', 9);
162     end
163 end
164 sgtitle('Pole-Zero Maps for Different a Values');
165 saveas(gcf, 'results/figures/Task02_Figure_01.png');
166
167 % --- Step Response Comparison ---
168 figure(2);
169 hold on;
170 colors = {'b', 'r', 'g'};
171 for idx = 1:length(a_values)
172     a = a_values(idx);
173     sys_tf = tf([1, a], [1, 10, 27, 8]);
174     step(sys_tf, 10);
175 end
176 hold off;
177 title('Step Response Comparison');
178 xlabel('Time (seconds)');
179 ylabel('Amplitude');
180 legend('a = -2', 'a = 0', 'a = +2', 'Location', 'best');
181 grid on;
182 saveas(gcf, 'results/figures/Task02_Figure_02.png');
183
184 % --- Bode Diagram Comparison ---
185 figure(3);
186 hold on;
187 for idx = 1:length(a_values)
188     a = a_values(idx);
189     sys_tf = tf([1, a], [1, 10, 27, 8]);
190     bode(sys_tf);
191     hold on;
192 end
193 hold off;
194 title('Bode Diagram Comparison for Different a Values');
195 legend('a = -2', 'a = 0', 'a = +2', 'Location', 'southwest');
196 grid on;

```

```

197 saveas(gcf, 'results/figures/Task02_Figure_03.png');
198
199 fprintf('\nTask 2 completed successfully.\n');
200 diary off;

```

5.3. Task 03: 采样率对离散化的影响

```

1  % Sampling Rate Effects on Frequency Response Analysis
2  clc;
3  clear;
4  close all;
5  diary('results/logs/Task03_output.log');
6
7  fprintf('
      =====\n
      n');
8  fprintf('Task 3: Sampling Rate Effects on Frequency Response\n');
9  fprintf('
      =====\n
      n');
10
11 %  $G_2(s) = 3 / (s^3 + 5s^2 - 3s)$  (from Task 1, part 2)
12 num_G2 = [3];
13 den_G2 = [1, 5, -3, 0];
14 G2 = tf(num_G2, den_G2);
15
16 fprintf('Continuous system:  $G_2(s) = 3/(s^3 + 5s^2 - 3s)$ \n\n');
17
18 % --- Find Gain Crossover Frequency ---
19 w_range = logspace(-3, 3, 10000);
20 [mag, ~, w_out] = bode(G2, w_range);
21 mag_dB = 20*log10(squeeze(mag));
22
23 % Find the 0 dB crossing (gain crossover frequency)
24 cross_idx = find(mag_dB(1:end-1) >= 0 & mag_dB(2:end) < 0, 1);
25 if isempty(cross_idx)
26     cross_idx = find(mag_dB(1:end-1) <= 0 & mag_dB(2:end) > 0, 1);
27 end
28
29 if ~isempty(cross_idx)
30     w1 = w_out(cross_idx);
31     w2 = w_out(cross_idx + 1);
32     m1 = mag_dB(cross_idx);
33     m2 = mag_dB(cross_idx + 1);
34     wc = w1 + (0 - m1) * (w2 - w1) / (m2 - m1);
35 else
36     [~, min_idx] = min(abs(mag_dB));
37     wc = w_out(min_idx);

```

```

38     fprintf('Warning: No clear 0 dB crossing found. Using minimum
        magnitude frequency.\n');
39 end
40
41 f1 = wc / (2 * pi);
42 fprintf('Gain crossover frequency:\n');
43 fprintf('  wc = %.6f rad/s\n', wc);
44 fprintf('  f1 = %.6f Hz\n', f1);
45
46 % Also compute using margin function for comparison
47 [Gm_c, Pm_c, Wcg_c, Wcp_c] = margin(G2);
48 fprintf('\nMargin function results (continuous):\n');
49 fprintf('  Gain crossover freq (Wcp) = %.6f rad/s\n', Wcp_c);
50 fprintf('  Phase crossover freq (Wcg) = %.6f rad/s\n', Wcg_c);
51 fprintf('  Gain margin = %.2f dB\n', 20*log10(Gm_c));
52 fprintf('  Phase margin = %.2f deg\n', Pm_c);
53
54 % Use the margin function's crossover frequency if available
55 if ~isnan(Wcp_c) && Wcp_c > 0
56     wc = Wcp_c;
57     f1 = wc / (2 * pi);
58     fprintf('\nUsing margin function crossover frequency: f1 = %.6f Hz
        \n', f1);
59 end
60
61 % --- Sampling Rates ---
62 fs_multipliers = [1, 3, 10];
63 num_rates = length(fs_multipliers);
64
65 w_bode = logspace(-2, 3, 800);
66
67 fprintf('\n
        =====\n
        n');
68 fprintf('Discretization and Analysis\n');
69 fprintf('
        =====\n
        n');
70
71 margin_results = zeros(num_rates, 4);
72
73 for i = 1:num_rates
74     fs = fs_multipliers(i) * f1;
75     Ts = 1 / fs;
76     fn = fs / 2;
77     wn = fn * 2 * pi;
78
79     fprintf('\n--- Case %d: f = %d * f1 (Ts = %.6f s) ---\n', ...
80         i, fs_multipliers(i), Ts);

```

```

81     fprintf(' Sampling frequency fs = %.6f Hz\n', fs);
82     fprintf(' Nyquist frequency fn = %.6f Hz (%.6f rad/s)\n', fn, wn)
      ;
83
84     Gz = c2d(G2, Ts, 'zoh');
85     fprintf(' Discrete transfer function Gz(z):\n');
86     disp(Gz);
87
88     figure(i);
89     subplot(2, 1, 1);
90     if fs_multipliers(i) <= 3
91         w_display = logspace(-2, log10(wn * 1.2), 800);
92         bode(G2, w_display);
93         hold on;
94         bode(Gz, w_display);
95         hold off;
96     else
97         bode(G2, w_bode);
98         hold on;
99         bode(Gz, w_bode);
100        hold off;
101    end
102    title(sprintf('Bode Diagram: f_s = %d * f_1 = %.4f Hz (T_s = %.6f
      s)', ...
103        fs_multipliers(i), fs, Ts));
104    legend('Continuous G2(s)', 'Discrete Gz(z)', ...
105        'Location', 'southwest');
106    grid on;
107
108    subplot(2, 1, 2);
109    nyquist(Gz);
110    title(sprintf('Nyquist Diagram: Discrete System (f_s = %d * f_1)',
      fs_multipliers(i)));
111    grid on;
112
113    saveas(gcf, sprintf('results/figures/Task03_Figure_%02d.png', i));
114
115    [Gm_d, Pm_d, Wcg_d, Wcp_d] = margin(Gz);
116
117    fprintf('\n Frequency Response Margins:\n');
118    if ~isnan(Gm_d) && ~isnan(Pm_d)
119        fprintf(' Gain margin (discrete) = %.2f dB\n', ...
120            20*log10(Gm_d));
121        fprintf(' Phase margin (discrete) = %.2f deg\n', Pm_d);
122        margin_results(i, :) = [20*log10(Gm_d), Pm_d, Wcg_d, Wcp_d];
123    else
124        fprintf(' Margin computation returned NaN.\n');
125        margin_results(i, :) = [NaN, NaN, NaN, NaN];
126    end

```

```

127
128 [mag_c, ph_c] = bode(G2, wc);
129 [mag_d, ph_d] = bode(Gz, wc);
130 fprintf('\n Accuracy at crossover wc = %.4f rad/s:\n', wc);
131 fprintf(' Continuous: |G(jwc)| = %.4f (%.2f dB), ', ...
132         squeeze(mag_c), 20*log10(squeeze(mag_c)));
133 fprintf('phase = %.2f deg\n', squeeze(ph_c));
134 fprintf(' Discrete: |Gz(e^{jwcTs})| = %.4f (%.2f dB), ', ...
135         squeeze(mag_d), 20*log10(squeeze(mag_d)));
136 fprintf('phase = %.2f deg\n', squeeze(ph_d));
137 fprintf(' Magnitude error = %.4f dB\n', ...
138         abs(20*log10(squeeze(mag_c)) - ...
139             20*log10(squeeze(mag_d))));
140 fprintf(' Phase error = %.4f deg\n', ...
141         abs(squeeze(ph_c) - squeeze(ph_d)));
142 end
143
144 % --- Summary and Analysis ---
145 fprintf('\n
146         =====\n');
147 fprintf('SUMMARY: Sampling Rate Effects\n');
148 fprintf('
149         =====\n');
150 fprintf(' Sampling Rate | Ts (s) | Gain Margin (dB) | Phase
151         Margin (deg)\n');
152 fprintf('
153         -----+-----+-----+-----\n');
154 fprintf(' Continuous | N/A | %.2f | %.2f\n',
155         ...,
156         20*log10(Gm_c), Pm_c);
157 for i = 1:num_rates
158     fprintf(' f = %d * f1 | %.6f | %.2f | %.2f\n',
159             ...,
160             fs_multipliers(i), 1/(fs_multipliers(i)*f1), ...
161             margin_results(i, 1), margin_results(i, 2));
162 end
163 fprintf('
164         -----+-----+-----+-----\n');
165
166 fprintf('\n--- Effect Analysis ---\n');
167 fprintf('1. At f = f1 (lowest sampling rate):\n');
168 fprintf(' - Sampling frequency equals crossover frequency.\n');
169 fprintf(' - Significant aliasing and distortion expected.\n');
170
171 fprintf('\n2. At f = 3*f1:\n');

```

```

165 fprintf('    - Nyquist frequency exceeds crossover frequency.\n');
166 fprintf('    - Basic Nyquist criterion satisfied.\n');
167
168 fprintf('\n3. At f = 10*f1:\n');
169 fprintf('    - High sampling rate provides good approximation.\n');
170 fprintf('    - Phase lag from ZOH is minimal.\n');
171
172 fprintf('\n--- Conclusion ---\n');
173 fprintf('Higher sampling rates yield more accurate frequency response\n    matching.\n');
174 fprintf('A sampling rate of at least 10 times the crossover frequency\n    is recommended\n');
175 fprintf('for accurate digital control system implementation.\n');
176
177 fprintf('\nTask 3 completed successfully.\n');
178 diary off;

```